# File Comparison Report

Generated by t0900 on 20-May-2026

## Files

| | Left File | Right File |
|---|---|---|
| **File name** | gilbreathDF.m | gilbreathDF_thesis.m |
| **File path** | C:\Users\t0900\Documents\MATLAB\Volkan | C:\Users\t0900\Documents\MATLAB\Volkan |
| **Last modified** | 20-May-2026 11:20:11 | 20-May-2026 13:38:11 |

## Environment

**MATLAB** 26.1 (R2026a)

## Comparison Results

+ Insertion − Deletion ≠ Modification

| Left File | Right File |
|---|---|
| `%% Reference` | |
| `%  Gregory P. Gilbreath, Captain, USAF, Jan 26, 2001` | |
| `%` | |
| `%  "Prediction of Pilot-Induced Oscillations (PIO)` | |
| `%   Due to Actuator Rate Limiting Using the` | `%This M-file will calculate the closed-loop describing function` |
| `%   Open-Loop Onset Point (OLOP) Criterion"` | |
| `%` | |
| `%  Thesis` | |
| `%  AFIT/GAE/ENY/01M-02` | |
| `%` | |
| `%  Department of the Air Force` | |
| `%  Air University` | |
| `%  Air Force Institute of Technology` | |
| `%  Wright-Patterson Air Force Base, Ohio` | |
| `%` | |
| `%  Closed-loop describing function for the aircraft model from AIAA-95-3204-CP (Duda / Hanke / Gilbreath, OLOP criterion for the rate-limited PIO problem).` | `%for the aircraft model example from AIAA-95-3204-CP` |
| `%` | |
| `%  Fixes vs. original:` | |
| `%    1. eqs() referenced dfuncmag / dfuncphase, which were never defined.` | |
| `%       It now calls dfunction() once and uses both outputs (keeps` | |
| `%       magnitude and phase on the same spline branch).` | |
| `%    2. bode() outputs at a single frequency are squeeze()d to scalars.` | |
| `%    3. Dead "phi2(1);" statement removed.` | |

```
%    4. First discontinuity is reported once (not overwritten each fail).
%    5. Arrays pre-allocated.
%    6. Nichols plot labelled.
%    7. Figure(1) is plotted with using nicholsplot instead of plot function.
%    8. The graphs of figure(1) have been aligned to 0 dB -180 degrees.

clear; clc ;close all
global w R K magGc magGac phGc phGac dtr qco z

n = 1000; % Resolution

% System input frequency range, amplitude, rate limit
w   = logspace(-1, 2, n);
% qco = 1.1; % original value given in Gilbreath's paper
qco = 1;
R   = 60;

% System numbers
K   = 13.68;
Gc  = tf( 5.21 * conv([1 -57.36], conv([1 4.26], [1 0.55])), conv([1 2*0.442*22.85 22.85^2], conv([1 0], [1 1.16])) );
Gac = tf(-10.524 * conv([1 1.562], conv([1 0.038], [1 0])), conv([1 2*0.212*0.088 0.088^2], conv([1 3.75], [1 -1.44])) );

% Linear closed loop  pcl = K*Gc / (1 + Gc*Gac)
pcl = K*Gc / (1 + Gc*Gac);
```

```
clear all
global w R K magGc magGac phGc phGac dtr qco z
%System input frequency range, amplitude, rate limit
w=logspace(-1,2,100); qco=1.1; R=60;

%System numbers
K=13.68;
Gc=tf(5.21*conv([1 -57.36],conv([1 4.26],[1 .55])),conv([1 2*.442*22.85 22.85^2],conv([1 0],[1 1.16])));
Gac=tf(-10.524*conv([1 1.562],conv([1 .038],[1 0])),conv([1 2*.212*.088 .088^2],conv([1 3.75],[1 -1.44])));
num=K*Gc;
den=zpk([],[],1)+series(Gc,Gac);
pcl=num/den;
dtr=pi/180;

%Find linear response amplitude and phase for initial guess
[magpcl,phi2(1)]=bode(pcl,w(1));

deltao(1)=qco*magpcl; phi2(1);

for z=1:length(w)
    [magGc,phGc]=bode(Gc,w(z));
    [magGac,phGac]=bode(Gac,w(z));
```

```
dtr = pi/180;

% Pre-allocate
N       = numel(w);
% Number of frequency points in the logarithmic sweep
deltao = zeros(1, N+1);
% Surface-command amplitude at the rate-limiter input
phi2   = zeros(1, N+1);
% Phase of the surface command, in degrees
fval   = zeros(1, N);
% Converged residual of the harmonic-balance equation
NoMag  = zeros(1, N);
% Open-loop describing-function magnitude in dB
NoPh   = zeros(1, N);
% Open-loop describing-function phase in radians

% Find linear response amplitude and phase for initial guess
[magpcl, p0] = bode(pcl, w(1));
deltao(1)     = qco * squeeze(magpcl);
phi2(1)       = squeeze(p0);

onset_freq = NaN;
% The OLOP Frequency


% The loop is for per-frequency harmonic-balance solve

for z = 1:N

    % Frequency response evaluation

    [mGc,  pGc ] = bode(Gc,  w(z));
    [mGac, pGac] = bode(Gac, w(z));
```

```
    xo=[deltao(z) phi2(z)]; %initial guess for optimization algorithm

    [a,fval(z)]=fminsearch('eqs',xo,optimset('Display','off'));

    deltait=a(1);
    phi2it=a(2);

    [magNit,phNit]=dfunction(w(z),R,deltait);
    A=deltait*magGac*magNit/(K*qco);
    phi=phi2it+phGac+phNit*180/pi;

    %Calculate Open-loop describing function for Nichols plot
    NoMag(z)=20*log10(A/sqrt(1-2*A*cos(dtr*phi)+A^2));
    NoPh(z)=dtr*phi-atan2(-A*sin(dtr*phi),1-A*cos(dtr*phi));

```

```
    magGc  = squeeze(mGc);    phGc  = squeeze(pGc);   % squeeze collapses the 1×1×1 outputs to scalars
    magGac = squeeze(mGac);  phGac = squeeze(pGac);  % squeeze collapses the 1×1×1 outputs to scalars

    % The Harmonic-balance Solve

    xo = [deltao(z), phi2(z)];

    [a, fval(z)] = fminsearch(@eqs, xo, optimset('Display','off'));

    % Describing-function evaluation at the operating point

    deltait = a(1);
    phi2it  = a(2);

    [magNit, phNit] = dfunction(w(z), R, deltait);
    A   = deltait * magGac * magNit / (K * qco);
    phi = phi2it + phGac + phNit*180/pi;

    % Closed-loop -> open-loop describing function
    NoMag(z) = 20*log10( A / sqrt(1 - 2*A*cos(dtr*phi) + A^2) );
    NoPh(z)  = dtr*phi - atan2(-A*sin(dtr*phi), 1 - A*cos(dtr*phi));

    if fval(z) > 1e-4
        if isnan(onset_freq)
            onset_freq = w(z);
            fprintf('Discontinuity / onset frequency: %.4g rad/s\n', onset_freq);
        end
        deltao(z+1) = 24;       % The values tuned for this specific aircraft
        phi2(z+1)   =  -237;      % The values tuned for this specific aircraft
    else
        deltao(z+1) = a(1);
```

```
    %Give new initial guess if function cannot converge to zero
    %Disply onset frequency where discontinuity starts to occur
    if fval(z)>.0001
        deltao(z+1)=24; phi2(z+1)=-237;onset=w(z)
    else
        %Else, use previous solution for next starting point
```

```
        phi2(z+1)=a(2);
    end
end

%% Figure 1: Nichols chart aligned on (-180 deg, 0 dB)

% Wrap the manually-computed OLOP describing function (magnitude in dB, phase in rad) into an frd so nicholsplot can handle it.
H_dol   = (10.^(NoMag/20)) .* exp(1i * NoPh);
sys_dol = frd(H_dol(:), w, 'FrequencyUnit', 'rad/s');

% Linear open loop as frd on the same grid
H_lin   = squeeze(freqresp(Gc*Gac, w));
sys_lin = frd(H_lin, w, 'FrequencyUnit', 'rad/s');

% Plot options: wrap phase into [-360, 0] so it brackets -180
opts                           = nicholsoptions;
opts.PhaseMatching            = 'on';
opts.PhaseMatchingFreq        = 1;              % rad/s
opts.PhaseMatchingValue       = -180;           % anchor phase at -180 at w=1
opts.PhaseWrapping             = 'on';
opts.PhaseWrappingBranch       = -360;
opts.Grid                      = 'on';
opts.Title.String              = 'Nichols chart: linear loop vs. rate-limited OLOP DF';
opts.XLabel.String             = 'Open-loop phase';
opts.YLabel.String             = 'Open-loop magnitude';

figure(1);

hold on

```

```
        deltao(z+1)=a(1); phi2(z+1)=a(2);
    end
end

[mlin,plin]=bode(Gc*Gac,w);
figure(1) %Plot results on Nichols Chart
plot(plin(1,:)-360,20*log10(mlin(1,:)),'-o',NoPh/dtr,NoMag,'-*')
axis([-300,-50,-20,12])

figure(2) %Plot function evaluation vs. frequency to see convergence
semilogx(w,fval)

```

```
h = nicholsplot(sys_lin, sys_dol, opts);

% Crosshair at the critical point (kept out of the legend)

xline(-180, '--', 'Color', [0.35 0.35 0.35], 'LineWidth', 1, 'HandleVisibility','off');
yline(   0, '--', 'Color', [0.35 0.35 0.35], 'LineWidth', 1, 'HandleVisibility','off');
plot(-180, 0, 'r+', 'MarkerSize', 10, 'LineWidth', 1.5, 'MarkerFaceColor', 'r', 'HandleVisibility','off');

legend('Linear G_c G_{ac}', 'OLOP describing function', 'Location','best');

% Axis centered on the critical point (-180 deg, 0 dB)
xlim([-300 -50])
ylim([-20 20])

grid on

hold off

%% Figure 2: Convergence diagnostic

figure(2);

semilogx(w, fval);
grid on
xlabel('\omega [rad/s]');
ylabel('Residual f_{val}');
title('fminsearch residual vs. frequency');


%% THE LOCAL FUNCTIONS

function f = eqs(x)                          % The function of the harmonic balance residual
    global w R dtr magGc magGac phGc phGac K qco z
    deltao = x(1);
    phi2 = x(2);

```

```
function f=eqs(x)
global w R dtr magGc magGac phGc phGac K qco z

deltao=x(1); phi2=x(2);

```

```
    [magN, phN] = dfunction(w(z), R, deltao);    % magN: gain, phN: rad

    re = deltao/magGc * cos(dtr*(phi2 - phGc)) + deltao*magGac*magN * cos(dtr*(phi2 + phGac) + phN) - K*qco;

    im = deltao/magGc * sin(dtr*(phi2 - phGc)) + deltao*magGac*magN * sin(dtr*(phi2 + phGac) + phN);

    f = re^2 + im^2;
end

function [magN, phN] = dfunction(freq, rate, inamp)
% Describing function of a rate-limiting element (Duda piecewise form).
%   magN  - dimensionless gain  (1 below onset; 4/(pi*x) far above)
%   phN   - phase in RADIANS    (0 below onset; -acos(pi/(2x)) far above)
    x = freq * inamp / rate;
    if x < 1                       % Region I
        magN = 1;
        phN  = 0;
    elseif x < 1.862               % Region II
```

```
f=(deltao/magGc*cos(dtr*(phi2-phGc))+deltao*magGac*dfuncmag(w(z),R,deltao)...
    *cos(dtr*(phi2+phGac)+dfuncphase(w(z),R,deltao))-K*qco)^2.0...
    +(deltao/magGc*sin(dtr*(phi2-phGc))+deltao*magGac*dfuncmag(w(z),R,deltao)...
    *sin(dtr*(phi2+phGac)+dfuncphase(w(z),R,deltao)))^2.0;

%Function file that determines magnitude and phase of describing function given the input frequency
%rate limit and input amplitude. Includes cubic spline interpolation coefficients.
function [magN,phN]=dfunction(freq,rate,inamp)
%Describing Function of a rate limiting element
x=freq*inamp/rate;
if x<1, magN=1;phN=0;

elseif x<1.862
    magN=polyval([.2908 -1.4396 1.9232 .223],x);%From cubic spline interpolation
    phN=polyval([.528 -2.6213 3.5056 -1.4171],x);% From cubic spline interpolation
else
    magN=4/x/pi;
    phN=-acos(pi/x/2);
end
```

```
        magN = polyval([ 0.2908 -1.4396  1.9232  0.2230 ], x);
        phN  = polyval([ 0.5280 -2.6213  3.5056 -1.4171 ], x);
    else                          % Region III
        magN = 4 / (x*pi);
        phN  = -acos(pi/(2*x));
    end
end

```